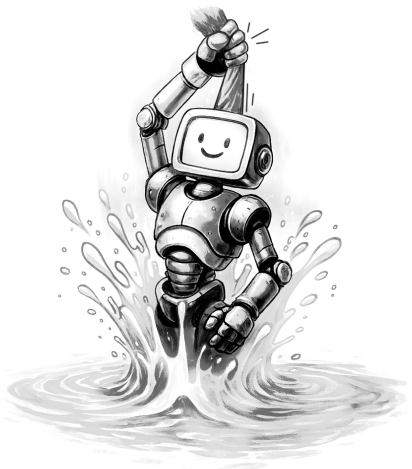
arXiv:2510.21614v3 [cs.AI] 29 Oct 2025

HUXLEY-GÖDEL MACHINE: HUMAN-LEVEL CODING AGENT DEVELOPMENT BY AN APPROXIMATION OF THE OPTIMAL SELF-IMPROVING MACHINE

Wenyi Wang* **Piotr Piękos*** **Li Nanbo** **Firas Laakom** **Yimeng Chen**
Mateusz Ostaszewski **Mingchen Zhuge** **Jürgen Schmidhuber**
{wenyi.wang, piotr.piekos, nanbo.li, firas.laakom, yimeng.chen,
mateusz.ostaszewski, mingchen.zhuge, juergen.schmidhuber}@kaust.edu.sa
King Abdullah University of Science and Technology (KAUST)
Thuwal, Saudi Arabia

ABSTRACT

Recent studies operationalize self-improvement through coding agents that edit their own codebases. They grow a tree of self-modifications through expansion strategies that favor higher software engineering benchmark performance, assuming that this implies more promising subsequent self-modifications. However, we identify a mismatch between the agent's self-improvement potential (metaproductivity) and its coding benchmark performance, namely the *Metaproductivity-Performance Mismatch*. Inspired by Huxley's concept of clade, we propose a metric (CMP) that aggregates the benchmark performances of the *descendants* of an agent as an indicator of its potential for self-improvement. We show that, in our self-improving coding agent development setting, access to the true CMP is sufficient to simulate how the Gödel Machine would behave under certain assumptions. We introduce the Huxley-Gödel Machine (HGM), which, by estimating CMP and using it as guidance, searches the tree of self-modifications. On SWE-bench Verified and Polyglot, HGM outperforms prior self-improving coding agent development methods while using fewer allocated CPU hours. Last but not least, HGM demonstrates strong transfer to other coding datasets and LLMs. The agent optimized by HGM on SWE-bench Verified with GPT-5-mini and evaluated on SWE-bench Lite with GPT-5 **achieves human-level performance, matching the best officially checked results of human-engineered coding agents**. Our code is publicly available at https://github.com/metauto-ai/HGM.



*equal contribution

1 INTRODUCTION

Processes of self-modification drive the growth of complex systems, from biological evolution (Hendrikse et al., 2007; Dawkins, 2019) to cultural and scientific innovation (Good, 1966; Hall, 2007). These general ideas have been instantiated in concrete algorithms for self-improving agents (Schmidhuber, 1987; 2003; Nivel et al., 2013; Everitt et al., 2016), demonstrating how abstract principles of self-modification can be translated into operational mechanisms. Unlike static systems constrained by fixed architectures, such agents can incrementally modify their own self-modification mechanisms and learning strategies, reusing newly gained abilities to fuel subsequent improvements. This capacity fosters continual adaptation, reduces reliance on human intervention, and enables problem-solving capabilities that cannot be fully anticipated at design time.

A central challenge is how to decide which self-modifications to accept. The Gödel machine (Schmidhuber, 2003) (GM) offers a theoretically optimal answer: accept only modifications that provably increase the expected long-term utility. While this provides a sound blueprint, its reliance on formal proofs makes it practically challenging. Recent implementations instead rely on coding agents that edit their own codebases and favor self-modifications from agents with higher benchmark performance (Robeyns et al., 2025; Zhang et al., 2025a). Yet, as illustrated in Figure 1 (left), this heuristic can be misleading: a high-scoring agent may produce unproductive descendants, while a lower-scoring one seeds lineages that achieve greater long-term gains. We term this phenomenon the *Metaproductivity–Performance Mismatch*.

To address this mismatch, we introduce *clade-level metaproductivity* (CMP), inspired by Huxley's notion of clades as lineages of common ancestry (Huxley, 1957). CMP quantifies the productivity of a clade by aggregating the success of an agent's descendants rather than relying solely on its immediate benchmark score. Furthermore, we show in Theorem 1 that in our self-improving coding agent development setting (Assumption 1, which includes the assumption that the only quality of the self-improvement process is the evaluation score of the final agent and that the evaluation is conducted with repeatable trials), having access to the true CMP oracle suffices to imitate the Gödel Machine.

This insight motivates our proposed algorithm, the **Huxley–Gödel Machine (HGM)**, which approximates GM-style self-improvement by estimating CMP from clade-aggregated descendant outcomes and selecting nodes to expand via Thompson sampling. Furthermore, by leveraging a more reliable estimate, we adaptively decouple expansion from evaluation, leading to asynchronous execution for efficient parallelism.

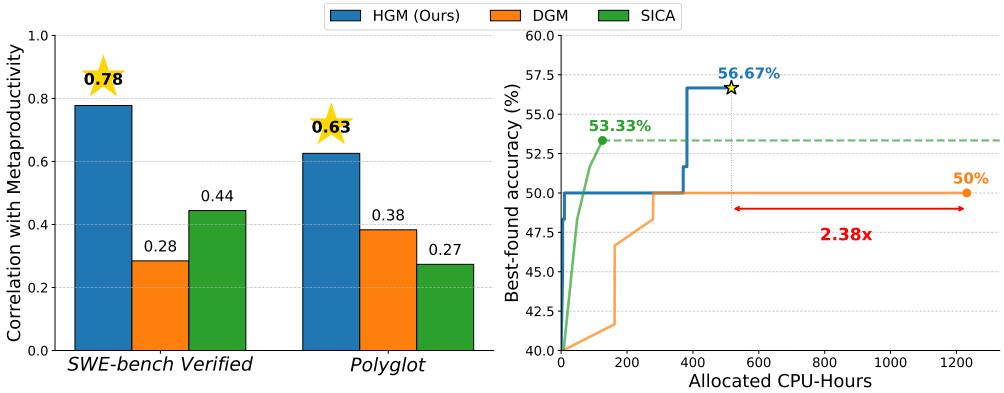


Figure 1: **(Left)** Weak correlation between the guidance metrics of other methods (based on benchmark performance) and long-term self improvement; HGM mitigates this mismatch by leveraging clade-level metaproductivity. **(Right)** On *SWE-bench Verified*, HGM achieves **higher** accuracy with 2.38 time less allocated CPU-hours. Together, the results indicate the practical advantage of approximating Gödel Machines with long-term self-improvement estimates. Note that SICA encountered repeated errors after consuming 45% of its budget, preventing any further self-modifications.

Empirically, HGM better aligns with long-run agent productivity than benchmark-driven baselines, as shown in Figure 1 (left). On SWE-bench Verified (Jimenez et al., 2024; Chowdhury et al., 2024) and Polyglot (Gauthier, 2024), HGM consistently outperforms Darwin Gödel Machine (DGM) (Zhang et al., 2025a) and Self-Improving Coding Agent (SICA) (Robeyns et al., 2025). Remarkably, one agent found by HGM surpasses SWE-agent (Yang et al., 2024), the highest-scoring human-engineered coding agent with officially checked results, on SWE-bench Lite (Jimenez et al., 2024), when both use the GPT-5-mini backbone under matched budgets. The HGM-discovered agent transfers robustly when evaluated under a shift that is simultaneous in both the dataset and the model. Although optimized on SWE-bench Verified with GPT-5-mini, when tested on SWE-bench Lite with the GPT-5 backbone, **it achieves performance on par with the best officially verified human-engineered coding agents**.

To summarize, our contributions are as follows:

- We analytically define the Clade-Metaproductivity (CMP) function as a measure of agents' self-improving ability and show that in a self-improving coding agent development setting (Assumption 1), access to a CMP oracle suffices to reproduce the Gödel Machine's acceptance mechanism. (Theorem 1).
- We empirically observe that immediate benchmark performance is an unreliable predictor of CMP and show that our CMP estimator aligns better.
- Using our CMP estimator, we propose the **Huxley–Gödel Machine (HGM)**, which approximates the Gödel Machine in a coding agent setting from partial evaluations and guides the expansion via Thompson sampling with adaptive scheduling.
- We empirically validate HGM on SWE-bench Verified and Polyglot, demonstrating higher-quality optimized agents compared to previous self-improving methods, even though they were discovered within substantially smaller allocated CPU-hours. Furthermore, HGM **achieves human-level coding agent design** on SWE-bench Lite by optimizing on SWE-bench Verified.

2 Self-Improvement as Tree-Search

Both the Darwin Gödel Machine (DGM) and the Self-Improving Coding Agent (SICA) belong to the class of self-referential AI (Schmidhuber, 1987; 2003). In particular, in DGM and SICA, agents modify themselves to generate new agents, each empirically validated on downstream tasks.

In this paper, we formalize this self-improvement process as an iterative tree-search problem, where the goal is to discover an agent that maximizes performance across multiple downstream tasks. Concretely, starting from an initial agent as the root, a tree-search policy incrementally grows the tree of self-modified agents. At each iteration, the policy either selects an agent (a node in the tree) to expand by producing a child agent (a self-modified version of the selected agent) or selects an agent to undergo additional evaluation on downstream tasks.

Formally, let $\mathcal{T}_t$ denote the archive of our agents at iteration t. In this paper, the archive is always represented as a tree of evolved agents, and we use the terms archives and trees interchangeably. $\mathcal{T}_0 = \{a_0\}$ is initialized as a single-node tree with a fixed initial agent. At each iteration t, the policy selects actions $a_{t+1} \sim \pi(\cdot \mid \mathcal{T}_t)$, where π is a policy over actions $\mathcal{A}_t = \mathcal{M}_t \cup V_t$, $\mathcal{M}_t = \{m_a : a \in \mathcal{T}_t\}$ are agent modifications, and $V_t = \{v_a : a \in \mathcal{T}_t\}$ denotes evaluations. Action m_a instructs the agent a to produce a self-modification that is added as a child of a to the tree, and v_a selects agent a from the tree for an additional evaluation on one more downstream task. After exhausting the computational budget, the policy selects a final agent ($a_{\text{final}} = \arg\max_{a \in \mathcal{T}} Score_\pi(a) \in \mathcal{T}_B$ where B is the termination iteration and $Score$ is part of the policy) from the final tree as the returned agent. The objective is to optimize $J(\pi) = \mathbb{E}[U(a_{\text{final}})]$, where U is a utility function that measures the performance of downstream tasks. In this work, we consider U the average of binary success indicators across all downstream tasks. π denotes an algorithm, with DGM, SICA, and our proposed HGM representing concrete instances.

Compound Policy. At each step of self-improvement, the system faces a compound decision: whether to expand the tree by generating new agents or to evaluate existing ones. This decision naturally decomposes into three sub-policies: (i) a *selection policy* that chooses between expansion and evaluation, (ii) an *expansion policy* that determines which parent to modify, and (iii) an *evaluation*

policy that selects an agent to test. Prior approaches, such as SICA and DGM, conflate these choices. They always expand a parent, create a child, and immediately evaluate that child on multiple tasks. This fixed sequence restricts flexibility: once a new agent is generated, it monopolizes evaluations, even if older agents appear more promising. For instance, an agent that fails nine tasks in a row continues to consume evaluations, while an older agent with partial successes is ignored.

use only one metric to both evaluate and select an agent for expansion

HGM breaks this rigidity by *decoupling* expansion from evaluation. At each step, it adaptively decides whether to generate a new agent or to further probe an existing one, and evaluations are always at the granularity of a *single agent–task pair*. This finer control enables early stopping on unpromising agents. Table 5 summarizes how SICA, DGM, and HGM instantiate these sub-policies.

i.e. by using one metrics for the selection of the agent to use for expansion and another to evaluate how good an agent is

3 HUXLEY-GÖDEL MACHINE

In this section, we introduce the Huxley–Gödel Machine (HGM), a self-improving machine that approximates Gödel Machine by using clade-level statistics. At the core of HGM lies the notion of metaproductivity—a measure of an agent's ability to improve its self-improvement skills, which leads to better downstream performance of distant future agents.

In Section 3.1 , we state the original Gödel machine and describe how our scope is specialized with respect to the general setup of the Gödel machine. In Section 3.2, we introduce two metrics of metaproductivity: Global metaproductivity (GMP), which captures how evolving a given agent increases the metaproductivity of the entire agent tree. This measure of metaproductivity is general and difficult to operationalize or estimate. We instead introduce clade-metaproductivity (CMP), which measures how promising the evolutions starting from a given agent (its clade) are. In Theorem 1, we show that access to true CMP is sufficient to implement a Gödel Machine applied to the coding agent development setting (Assumption 1). Following on that, in Section 3.3, we introduce the Huxley-Gödel Machine (HGM), which guides the self-improvement search with Thompson Sampling based on the estimate of CMP.

3.1 GÖDEL MACHINE IN THE SELF-IMPROVING CODING AGENT DEVELOPMENT SETTING

The original Gödel Machine is a general task solver that, in principle, can optimally make any provable self-improvements in any computable environment with respect to a given objective (Schmidhuber, 2003). It achieves this by running a proof searcher, continually looking for formal proofs that some modification of its own code will yield higher expected utility. Once such a proof is found, the modification is executed and permanently alters the machine. Importantly, the theoretical analysis of the Gödel Machine explicitly accounts for the fact that the agent has only a single life (with no repeatable trials) and that proving a self-improvement consumes real time and resources, which could be used for gathering the reward.

In contrast, this paper focuses on problems in a setting tailored to a particular kind of self-improving coding agent development that follows Assumption 1. Specifically, our problem assumes that the only objective is the utility of the agent selected at the end of the agent development process. Moreover, in our problem, the tests used to evaluate coding agents are conducted in a repeatable manner. This means that the testing environment is reset for each test, and prior evaluations do not influence later test results. Furthermore, each self-modification reduces the remaining budget by exactly one unit. For the theoretical analysis, we also assume that the only operation that incurs a time cost is the self-modification itself.

Within this framework designed for self-improving coding agent development, the Gödel Machine is an optimal agent operating in a POMDP where the policy observes only a parent a_{parent}, its child a_{child}, and the remaining budget b, and then chooses to accept or reject the child. At termination, $Score_\pi$ selects either the final parent or child as output. Full description of the POMDP can be found in the Appendix A. We clarify the additional assumptions in our setup in comparison to the general Gödel Machine in Assumption 1.

Assumption 1. *For the theoretical analysis of Gödel Machine applied to self-improving coding agents, we make the following additional assumptions in comparison to the setup from the original Gödel Machine:*

- *The policy objective function is defined as a function of only the final agent, with no other rewards received before termination;*
- *The agent's utility is measured by its performance on evaluation tasks, under the assumption of repeatable trials: for any agent–task pair, the expected outcome is independent of evaluation time or prior events.*
- *The proofs of Gödel Machines do not consume budget;*
- *And each self-modification costs exactly one unit of the budget.*

3.2 METAPRODUCTIVITY AND CLADE-METAPRODUCTIVITY

The global metaproductivity measures the impact of self-improvement on the entire current tree archive of all agents. Given a policy π, to quantify the quality of how an agent's self-modification influences the performance of the system, we define the notion of global metaproductivity (GMP):

$$\mathrm{GMP}_\pi(\mathcal{T}, a) \;=\; \mathbb{E}_{\mathcal{T}_B \sim p_\pi(\cdot|\mathcal{T},a)} \left[U(\mathrm{argmax}_{a' \in \mathcal{T}_B} Score_\pi(a')) \right],$$

where $\mathcal{T}$ is a tree of agents and $a \in \mathcal{T}$. $Score_\pi$ is the function that scores the agents for the final selection. The policy π unrolls the trajectory until the end of the episode with policy π and produces a final archive of agents $\mathcal{T}_B$. The distribution of the trajectory is given by p_π.

GMP directly corresponds to the Q-value function in reinforcement learning, with the state phrased as the archive of agents and the action being the selected agent to expand. The GMP value of a node measures how well (in expectation) the final agent obtained from the search process will perform. GMP measures the long-term potential of self-improvement for the entire tree, which also includes modifications that improve self-improvement itself, and so on. An algorithm might, at the beginning, focus on improving the ability to self-improve while neglecting direct benchmark abilities, only to later focus on them. This is a principal meta-learning behavior that is captured in the original Gödel Machine (Schmidhuber, 2003). The objective of designing a policy for self-improvement (Section 2) is equivalent to optimizing $\mathrm{GMP}(\{a_0\}, a_0)$.

While GMP captures the full global potential of a policy with access to the history of agents, its scope is too broad for practical conceptualization, since, in principle, the self-modifications of an agent can influence the expected utility of a parent agent by introducing new information. The Gödel Machine achieves global-optimality by deciding whether to accept or reject self-modification, focusing on the provable potential subsequent self-improvements. Motivated by this observation, we define a localized variant of GMP that focuses on the subtree rooted at a given agent, i.e., its *clade*. We refer to this quantity as Clade-Metaproductivity (CMP):

$$\begin{aligned}
\mathrm{CMP}_\pi(\mathcal{T}, a) &= \mathbb{E}_{\mathcal{T}_B \sim p_\pi(\cdot|\mathcal{T},a)} \left[U(\mathrm{argmax}_{a' \in C(\mathcal{T}_B,a)} Score_\pi(a')) \right] \\
&= \mathbb{E}_{\mathcal{T}_B \sim p_\pi(\cdot|\mathcal{T},a)} \left[\mathrm{max}_{a' \in C(\mathcal{T}_B,a)} U(a') \right] \qquad (\text{if } Score = U),
\end{aligned}$$

where $C(\mathcal{T}_B, a)$ is the clade (i.e., the subtree with a as the root) of the node a in the Tree $\mathcal{T}_B$ and $Score$ is the final agent selection metric. CMP contains the non-greedy information about the future evolution of self-improving agents, therefore guiding good strategies for self-improvement aimed also at the improvement of self-improvement itself. Furthermore, we show the crucial relation of CMP to the Gödel Machine.

Theorem 1. *Under Assumption 1, access to the* CMP *oracle is sufficient to implement the Gödel Machine.*

The proof is available in the App. A. This observation motivates us to introduce the estimate of CMP and use this as guidance in our algorithm. By estimating CMP, HGM approximates the Gödel Machine. We describe our algorithm fully in the next section.

3.3 ALGORITHM

Existing methods use benchmark performance on coding tasks as a guiding metric, treating task success as an indicator of self-improvement potential. This assumption is overly greedy: it evaluates only the immediate utility of a modification while ignoring its downstream consequences for future self-modifications. We refer to this gap as the *Metaproductivity-Performance Mismatch*: the divergence between short-term task performance and the long-term capacity for self-improvement, as measured by CMP. Empirical evidence shows that this mismatch occurs in practice (see Section 4.1.) We aim to model long-term, global dependencies by deriving our estimator of CMP. Specifically, we define HGM by stating its three subpolicies.

Expansion Policy. The core of the HGM algorithm is its selection criterion for expansion. HGM aims to estimate Clade-Metaproductivity with the motivation that, in our setting, under Assumption 1, the true CMP as the criterion would produce the Gödel Machine due to Theorem 1. In this sense, HGM approximates Gödel Machine, the **optimal** self-improving machine. This is in contrast to the currently used greedy selection criteria based on performance metrics, which ignore the potential of the model to improve its self-improving abilities.

We estimate CMP using the **weighted** average of agents' empirical performance in the clade. HGM is designed to promote higher weights to higher utility agents. See below for how our evaluation policy induces this weighting strategy. Formally, let us assume a fixed archive of agents $\mathcal{T}_t$, $n_{\text{success}}(a)$ be the number of passed tests of a, and $n_{\text{failure}}(a)$ be the number of failed tests of a. Then

$$n^C_{\text{success}}(a) = \sum_{a' \in C(a)} n_{\text{success}}(a') \quad \text{and} \quad n^C_{\text{failure}}(a) = \sum_{a' \in C(a)} n_{\text{failure}}(a'),$$

where $C(a)$ is the clade of a in $\mathcal{T}_t$. We define our Clade-Metaproductivity estimator as

$$\widehat{\text{CMP}}(a) = \frac{n^C_{\text{success}}(a)}{n^C_{\text{success}}(a) + n^C_{\text{failure}}(a)}.$$

Evaluating productivity at the level of entire clades rather than individual agents offers several key advantages. It aligns better with the goal of self-improvement, as a modest ancestor can still be highly valuable if its descendants consistently advance, while stagnant lineages are deprioritized. At the same time, aggregating evidence across a clade yields more statistically robust estimates than single-node outcomes by using information from more samples. This is particularly important when evaluations are costly and benchmarks are only partially observed.

$\widehat{\text{CMP}}(a)$ can be viewed as a weighted sum over the empirical means of agents in $C(a)$, with the weight for an agent being the number of task evaluations it has performed. Furthermore, we design our evaluation selection in such a way that it selects **highly performing agents**, which creates a selection of a soft maximum in the clade.

After calculating the CMP estimates, the HGM probabilistically approximates the selection of the highest scoring agent using Thompson Sampling—a standard method in the bandit literature for smoothly maximizing the decision criterion (Agrawal & Goyal, 2012; Chapelle & Li, 2011). We will refer to $a \sim TS(\{(n_s, n_f) | n \in \mathcal{T}_t\})$ as the agent sampled from the Thompson-Sampling process with parameters n_s (number of successes) and n_f (number of failures). Given that the search problem has a known budget, our algorithm introduces an exploration-exploitation scheduler τ that is monotonically increasing with respect to the current time t, encouraging exploration in the early stages and polarization of the sampling distribution as it approaches the end. Formally, we select the agent to expand a^* as

$$a^* \sim TS(\{(\tau(1 + n^C_{\text{success}}(a)), \tau(1 + n^C_{\text{failure}}(a))) | a \in \mathcal{T}_t\}).$$

Evaluation Policy. As stated in the expansion policy, we design our evaluation policy to prioritize **agents** with a higher evaluation score to induce the selection of the maximum over the clade. Formally, the agent to evaluate a^* is sampled from the Thompson Sampling process with

$$a^* \sim TS(\{(\tau(1 + n_{\text{success}}(a)), \tau(1 + n_{\text{failure}}(a))) | a \in \mathcal{T}_t\}.$$

Selection Policy. Finally, our agent has to choose between expansion and evaluation. At each iteration, the algorithm first selects whether to evaluate or expand. Previous methods have evaluated newly created agents directly after their creation. Our novel estimation of agent self-improving quality has the additional benefit of collecting more samples more quickly (because it has samples from the entire clade). This enables more fine-grained control over when to evaluate and when to create a new agent for better efficacy. Therefore, we decouple evaluation from expansion and treat them as separate steps.

To decide how and when to evaluate or expand agents, we draw inspiration from the infinite-armed bandit literature. Infinite-armed bandit problems capture the tension between repeatedly sampling known options to reduce uncertainty about promising arms and exploring new options that have the potential to perform better. This perspective provides a natural lens for our setting, where evaluations correspond to sampling existing arms, and expansions correspond to introducing new ones. In this work, we follow the strategy of UCB-Air (Wang et al., 2008), which adds arms when the number of evaluations $N^\alpha \geq m$ for some $\alpha \in [0, 1]$, where m is the number of existing arms. In our case, arms correspond to the agents; hence, we decide to expand at time t if $N_t^\alpha \geq |\tilde{\mathcal{T}}_t|$.

Final Agent Selection Strategy. HGM iteratively executes the structured policy defined by our selection policy, expansion policy, and evaluation policy. When the computational budget exceeds, it returns the agent with the highest ϵ percentile of the utility posterior in the final tree for some hyperparameter ϵ, namely the **best-belief agent**. Formally, a best-belief agent is defined as

$$\text{argmax}_{a \in \mathcal{T}_B} I_\epsilon(1 + n_{\text{success}(a)}, 1 + n_{\text{failure}}(a)),$$

where I is the regularized incomplete beta function. See Algorithm1 in Appendix B for the detailed procedure of HGM.

Asynchronous Implementation. As an additional benefit of decoupling the policy, we introduce asynchronous execution of evaluation and expansion. Since the execution of coding agents generally requires querying large language models multiple times, the computation time can be lengthy. To boost our algorithm, we propose the asynchronous HGM algorithm (HGM Async), which utilizes all available computational power until the computational budget is exceeded. HGM Async concurrently executes an iteration of the process on each available CPU. Once one iteration finishes, a new iteration immediately starts. It uses the most recent data, with one exception, and updates the data once it is finished. The exception is that one needs to take all running expansions and explorations into consideration when executing the selection strategy. See experimental results 2 for the runtime comparison with DGM and SICA.

4 Experimental Results

We evaluate HGM on challenging software engineering tasks to assess three core aspects: **1)** the fidelity of HGM's CMP estimation (Sec. 4.1), **2)** its capability for self-improvement with HGM compared with DGM and SICA (Sec. 4.2), and **3)** the effectiveness in automatic agent design through evolutionary processes, benchmarked against a leading human design up to date[1] (Sec. 4.3). We conducted our experiments on the SWE-bench Verified (SWE-Verified) and SWE-bench Lite (SWE-Lite) variants, as well as the Polyglot problems, both of which consist of coding challenges and are widely used for coding agent evaluation (Xia et al., 2025; Zhang et al., 2024; 2025b). We follow DGM's evaluation setting for Polyglot problems, where agents have no access to private test cases or test results. For budget considerations, in addition to the full datasets, we use 60-task subsets (SWE-Verified-60), derived from the first two stages of DGM's progressive evaluation. In all experiments, we employ HGM with an exploration-exploitation scheduler $\frac{B}{b}$, where b is the remaining budget, $\epsilon = 1$, and $\alpha = 0.6$. All experiments involving HGM use the HGM-Async algorithm. We apply an identical initial agent when compared to DGM and SICA, which is adopted from the official implementation of DGM. See Appendix C.1 for a detailed description of the initial agents used in different experiments.

[1]The leading SWE-agents on `https://www.swebench.com` (Lite) as of 24 October 2025.

Table 1: **Clade-Metaproductivity: Empirical vs. Estimation Correlation**. We report the Pearson correlations between the empirical $\mathrm{CMP}s$ and the estimates from DGM, SICA, and HGM on SWE-Verified-60 and Polyglot. For the weighted correlations, each prediction is weighted by the number of evaluations it has accessed.

Estimates	SWE-Verified-60		Polyglot	
	Weighted	Un-weighted	Weighted	Un-weighted
SICA	0.444	0.444	0.274	0.274
DGM	0.285	0.406	0.383	0.357
HGM (Ours)	**0.778**	**0.512**	**0.626**	**0.873**

4.1 Metaproductivity-Performance Mismatch

The experiments in this section are designed to serve two purposes: (i) to provide evidence of the Metaproductivity-Performance Misalignment (MPM) issue; and (ii) to assess whether the $\widehat{\mathrm{CMP}}$ of HGM is a more reliable CMP estimator than the utility measures adopted by DGM and SICA. To reveal the misalignment, we compute the correlation between their selection criterion and empirical CMP. To obtain empirical $\mathrm{CMP}s$, we analyze the expanded search tree after each method has completed its run. For every node in the tree, we define its empirical CMP as the maximum empirical mean of the task performance achieved within its clade, with the root of this clade excluded. This construction ensures that empirical CMP captures the self-improvement ability of a node. We exclude the root of a clade to avoid the circular use of the target in the estimators. For HGM, the $\widehat{\mathrm{CMP}}$ is defined as a function over the clade of a node; a critical adjustment is required to avoid target leakage. Specifically, we exclude the evaluations that are most directly related to the target: the root of the clade (an ancestor of the target) and the subtree rooted at a direct child of the evaluated node that contains the empirical maximum, thereby ensuring a fair comparison (see Appendix D for detailed computation). We report both the correlation coefficient weighted by the number of evaluations used in prediction, as well as the unweighted correlation. We conducted our experiments on the SWE-Verified-60 and Polyglot datasets.

Results & Discussion. Table 1 summarizes the correlations between the three estimators and the empirical $\mathrm{CMP}s$ (the targets). We first observe that the SICA and DGM estimators achieve positive Pearson correlation coefficients: 0.444 and 0.285 on SWE-Verified-60, and 0.274 and 0.383 on Polyglot, respectively, suggesting weak alignments, i.e., MPM. In comparison, HGM's estimator, $\widehat{\mathrm{CMP}}$, achieves substantially stronger weighted correlations of 0.778 and 0.626 on SWE-Verified-60 and Polyglot, respectively, as well as unweighted correlations of 0.512 and 0.8783, surpassing SICA and DGM by wide margins. These results provide strong indications that HGM, equipped with $\widehat{\mathrm{CMP}}$, offers a more reliable estimate of metaproductivity and effectively mitigates the MPM issues inherent to SICA and DGM.

4.2 Evaluating HGM Self-Improving Capability

To validate our hypothesis that our CMP estimator better predicts future self-improvement and, hence, leads to more effective self-modifications, we evaluate HGM against two state-of-the-art self-improving coding agent methods: DGM and SICA. We conduct controlled experiments under the same setup as DGM, with a budget-friendly modification applied consistently to both HGM and all baselines to ensure fairness, i.e., we adopt more cost-efficient backbone LLMs (GPT-5 for expansion and GPT-5-mini for evaluation for SWE-Verified; Qwen3-Coder-480B-A35B-Instruct for expansion and Qwen3-Coder-30B-A3B-Instruct for evaluation for Polyglot). For all methods, we start with the same initial ancestor, which achieves 40% and 20.3% on the SWE-Verified-60 and Polyglot, respectively. We compare the task performance of their best-belief final agents after a maximum allowance of 800 benchmark task evaluations, selected in DGM and SICA using empirical means. In addition, our asynchronous parallelization of expansion and evaluation enables self-improvement to consume fewer allocated CPU-hours than DGM and SICA (see Sec. 3.3). We verify this and report the allocated CPU-hours required for 800 evaluations.

Table 2: **Self-Improving Capability Comparison**. We report the task performance (in accuracy) of each method's best-belief agent and the allocated CPU-hours time required for 800 evaluations. Superscripted accuracies with "+" indicate performance gains over their respective initial agents.

Best-belief Agent of	SWE-Verified-60		Polyglot	
	Acc. (%)↑	Time (hours)↓	Acc. (%)↑	Time (hours)↓
SICA	50.0^{+10}	infinite loop	$25.4^{+5.1}$	572
DGM	$53.3^{+13.3}$	1231	$27.1^{+6.8}$	2385
HGM (Ours)	$\mathbf{56.7}^{+16.7}$	**517**	$\mathbf{30.5}^{+10.2}$	**347**

Results & Discussion. We summarize the comparison results in Table 2. Across both SWE-Verified-60 and Polyglot, all three methods successfully perform agent discovery by optimizing the initial agent through self-improvement. However, HGM's best-belief agent demonstrates not only the highest task performance—56.7% on SWE-Verified-60 and 30.5% on Polyglot—but also the best efficiency, requiring the fewest allocated CPU-hours for 800 evaluations: 6.86 times faster than DGM and 1.65 times faster than SICA on Polyglot, and 2.38 times faster than DGM on SWE-Verified-60. Notably, on SWE-Verified-60, SICA repeatedly encounters "query length out-of-LLM-context-window" during self-improvement processes after 360 evaluations. Despite this, the Polyglot results validate our hypothesis regarding HGM's runtime advantage over the baselines. In conclusion, HGM, equipped with a better utility estimator and asynchronous expansion–evaluation iterations, establishes itself as a more effective self-improving mechanism compared to DGM and SICA.

4.3 HGM vs. Humans: on Coding Agents Design

To gain a better understanding of its potential, we extend our evaluation of HGM by benchmarking it against the best human performance in coding agent design on SWE-Lite. We consider two settings: 1) optimization on full SWE-Verified and 2) generalization to SWE-Lite.

4.3.1 Optimization on Full SWE-Bench Verified

In this experiment, rather than using the SWE-Verified-60, we scaled the HGM evaluation to the full SWE-Bench Verified benchmark (500 coding challenges) with an increased number of HGM iterations (8000 evaluations). We further adjusted the initial agent so that it yields an improved accuracy of 53.2%. See Appendix C.1 for the adjustment details. Notably, this stronger starting point underscores the difficulty of further improvement due to a higher baseline. Despite this, HGM demonstrates significant gains and strong absolute performance.

Results & Discussion. After 8000 evaluations, HGM discovered an optimized agent that solves 61.4% tasks, surpassing the best human-designed agent built on GPT-5-mini on the SWE-Verified leaderboard. This establishes our discovered agent as the *top-scoring* GPT-5-mini–based system, and positions it among the *top-10* agents over all checked submissions, even when compared to systems built on stronger backbone models that can cost $5\times$ more (e.g., Claude-3.7). While higher scores on the leaderboard do not necessarily indicate superior general coding ability—since both human- and machine-designed agents may overfit to the benchmark—these results demonstrate a promising potential of HGM for competing with established human-designed baselines under identical model constraints.

4.3.2 Generalization To SWE-Bench Lite

To validate that HGM's self-evolution produces agents with stronger general coding ability, rather than merely overfitting to SWE-Verified, we evaluate the top agent discovered on SWE-Verified against unseen tasks. Specifically, we compare this agent with its initial ancestor (which achieved 53.2% on SWE-Verified) using SWE-Lite, a benchmark of 300 coding tasks, 93 of which overlap with SWE-Verified. For rigor and comparability, we report two settings: (i) a filtered setting where the 93 overlapping tasks are excluded, leaving only completely unseen tasks, and (ii) the full 300-task benchmark, identical to the standard evaluation used for human designs on the leaderboard. As

Table 3: **Generalization on SWE-Lite: HGM's Best-belief SWE-Verified Agent.** We report the accuracy of HGM's best-belief SWE-Verified agent on SWE-Lite under two settings: filtered (excluding tasks overlapping with SWE-Verified) and standard (the official leaderboard setting used for evaluating human-designed agents)).

Coding Agents	SWE-Lite Filtered (%)	SWE-Lite Standard (%)
HGM Initial Ancestor	34.8	44.0
SWE-agent+GPT-5-mini	39.6	47.6
HGM's Best-belief SWE-Verified Agent	**40.1**	**49.0**

Table 4: **Transfer to different LLMs on SWE-Lite: HGM's Best-belief SWE-Verified Agent.** Similarly, We report the accuracy of HGM's best-belief SWE-Verified (optimized with GPT-5-mini) agent on SWE-Lite (evaluated with GPT-5) under two settings: filtered (excluding tasks overlapping with SWE-Verified) and standard (the official leaderboard setting used for evaluating human-designed agents).

Coding Agents	SWE-Lite Filtered (%)	SWE-Lite Standard (%)
SWE-agent (Best on the LB)	**48.3**	56.7
HGM's Best-belief SWE-Verified Agent + GPT-5	47.8	**57**

of the time of writing, no checked submission using GPT-5-mini appears on the SWE-Lite leaderboard. To control for backbone differences and isolate agent design, we adapt the leading system (with checked submissions) (SWE-agent + Claude 4 Sonnet) by replacing its backbone with GPT-5-mini, yielding SWE-agent + GPT-5-mini, as an additional baseline for comparison.

Results & Discussion. We show the generalization results of HGM's best-belief SWE-Verified agent on SWE-Lite benchmark in Table 3. The best-belief HGM agent found on SWE-Verified achieves 40.1% under the filtered (completely unseen) setting and 49.0% under the standard setting. Compared to its initial ancestor (34.8% and 44.0%, respectively), these gains substantiate the effectiveness of HGM's self-evolution in improving general coding ability—rather than overfitting to the optimization set. Notably, the superior performance of our HGM agent achieved on the standard SWE-Lite places it firmly *in second place* on the SWE-Lite leaderboard among all checked submissions. Moreover, based on our local execution result of SWE-agent using the SWE-agent + Claude 4 Sonnet submission version with the same configuration, the agent optimized by HGM outperforms SWE-agent + GPT-5-mini, which achieves 39.6% (vs. 40.1% for us) on the filtered and 47.6% (vs. 49.0% for us) on the standard. This demonstrates that the edge arises not from the GPT-5-mini backbone, but from the genuine design improvements introduced by HGM evolution.

Transfer to bigger LLMs. We also examined how the discovered agent scales when paired with larger and better-performing LLMs. In particular, we replaced the GPT-5-mini backbone of HGM's Best-belief agent with the GPT-5 model to test whether the agent optimized with one LLM remains effective with another. The results indicate that the agent maintains its strong performance under this transfer. In particular, its accuracy on the SWE-Lite benchmarks is comparable to that of state-of-the-art, human-engineered coding agents reported on the leaderboard, suggesting that HGM's self-evolved design principles are reliably transferred across backbone sizes. The agent discovered by HGM with GPT outperforms all other agents on the SWE-Bench Lite leaderboard with officially checked results and is one task behind the best-performing model on our selected "SWE-bench Filtered". In Table 4, we compare it with the SWE-agent, which holds first place on the official SWE-Bench Lite leaderboard at the time of publication. This result further supports the conclusion that the improvements are due to genuine agent improvement rather than overfitting to a particular dataset or LLM.

5 Related Works

The general concepts of machine self-improvement were first systematically articulated by Good (1966), who described the possibility of "Intelligence Explosion" once machines acquire the capacity to design more capable successors. Early work on explicit self-improvements dates back to Schmidhuber (1987), which introduced self-referential learning mechanisms in which a system generates and evaluates modified descendant versions of itself. Follow-up work on self-improvement progressed through interaction and agentic reinforcement learning. The Success-Story Algorithm(SSA) (Schmidhuber & Zhao, 1996; Schmidhuber et al., 1997) progressively forces self-modifying policies to discover more effective self-modification strategies. Its core mechanism is based on hindsight: at each checkpoint, a sequence of self-modifications that did not yield higher long-term reward rates is systematically undone. In this way, SSA enforces continual improvement by ensuring that only those self-modifications associated with demonstrably greater reward intake per unit time are preserved. Fitness-Monotonic Execution (Kirsch & Schmidhuber, 2022a;b) reduces the outer-loop design by favoring the execution of models with higher ancestral performance. Meta-discovered update rules optimized optimizers (Metz et al., 2021) and black-box search (Lange et al., 2023). On the other hand, the Gödel Machine, a fully self-referential algorithm that rewrites its own code whenever it can *prove* an expected-utility improvement, provides a provably and globally optimal mechanism for self-improvement (Schmidhuber, 2003).

The rise of contemporary LLMs has created an opportunity to automate substantial aspects of software engineering. One concrete step in this direction is the development of coding agents, which extend LLMs with the ability to operate in conventional computing environments. ChatDev (Qian et al., 2023) first illustrated this idea in the context of automated bug fixing, and similar frameworks were later explored in SWE Yang et al. (2024), OpenHands (Wang et al., 2024), MetaGPT (Hong et al., 2024), and AgentLess (Xia et al., 2025).

The Self-Taught Optimizer (Zelikman et al., 2024) and Gödel Agent (Yin et al., 2024) first experimented with agents that modify their own scaffolding. Subsequently, DGM (Zhang et al., 2025a) and SICA (Robeyns et al., 2025) extend this direction by implementing self-modifying machines as full software engineering projects, where agents self-reference and modify their own repositories while validating changes through execution-grounded software engineering tasks. Both DGM and SICA, explicitly or implicitly, assume that higher software benchmark scores correspond to greater self-improvement capacity. In contrast, HGM introduces a qualitative measure of self-improvement consistent with the theoretical Gödel Machine and directs self-modifications using estimates of this measure.

The identified tree-search problem spans fixed-budget best-arm identification (BAI), Monte Carlo Tree Search, and infinite-armed bandits, introducing a distinct decision: explicit expansion actions that create new candidate leaves alongside ordinary evaluations. Fixed-budget BAI and Bayesian value-of-information methods assume a finite and known set of arms and offer guarantees for static candidates, thus not modeling the discovery of unknown arms (Audibert & Bubeck, 2010; Karnin et al., 2013; Frazier et al., 2008). Monte-Carlo Tree Search and its UCT variants (Coulom, 2006; Kocsis & Szepesvári, 2006) alternate selection, expansion, and simulation, while their backup and selection rules typically target cumulative reward rather than fixed-budget final-choice objectives under noisy, low-signal feedback, with limited guarantees for pure exploration of leaf quality (Kaufmann & Koolen, 2017). Infinite-armed bandit formulations capture the explore-discover tradeoff but typically model discoveries as i.i.d. draws from a reservoir, missing tree structure, and hierarchical dependencies (Wang et al., 2008; Bubeck et al., 2011; Carpentier & Valko, 2015).

6 Conclusion

In this work, we identify a key limitation in the search heuristics of current self-improving coding agents: Benchmark scores alone do not reliably indicate an agent's long-term potential for self-improvement, since high-scoring agents can still lead to stagnating lineages, while seemingly weaker ones may seed productive self-improvements. We refer to it as the Metaproductivity–Performance Mismatch. To address this gap, we introduce Clade-Metaproductivity (CMP), a lineage-based metric inspired by Huxley's notion of clades. We show that, under certain assumptions, when applied

to our self-improving coding agent search problem (Assumption 1), the CMP oracle is sufficient to implement the Gödel Machine (Theorem 1).

Building on this principle, we propose the Huxley–Gödel Machine (HGM), which approximates CMP and uses it to guide expansion through Thompson sampling with adaptive scheduling. Empirically, HGM consistently produces higher quality agents than prior self-improving frameworks while also reducing wall-clock time. Notably, HGM generalizes across both dataset and model shifts, achieving human-level coding agent design performance on SWE-bench Lite with GPT-5 despite being optimized on SWE-bench Verified with GPT-5-mini.

Taken together, these results suggest that clade-based measures of improvement potential, rather than immediate performance alone, lead to more effective forms of self-improvement. By demonstrating that clade-level evaluation can reliably guide the growth of coding agents, this work points to a new paradigm for the design of agentic improvement: one in which improvement is driven not by narrow benchmarks, but by the long-term generative potential of entire lineages. This perspective underscores the importance of systems that strengthen an agent's capacity to keep improving over time, rather than merely boosting their performance in the short term.

ACKNOWLEDGMENT

We thank Yuhui Wang for the discussions during the early stages of this project. We gratefully acknowledge Jenny Zhang and Shengran Hu, the authors of Darwin Gödel Machine, for sharing their insights about DGM and their implementation experience. We also thank Yilan Zhang, Rui Zhang, and Lisiyu Xie for their help in designing the visualizations. The research reported in this publication was supported by funding from King Abdullah University of Science and Technology (KAUST) - Center of Excellence for Generative AI, under award number 5940.

REFERENCES

Shipra Agrawal and Navin Goyal. Analysis of thompson sampling for the multi-armed bandit problem, 2012. URL https://arxiv.org/abs/1111.1797.

Jean-Yves Audibert and Sébastien Bubeck. Best arm identification in multi-armed bandits. In *COLT-23th Conference on learning theory-2010*, pp. 13–p, 2010.

Sébastien Bubeck, Rémi Munos, and Gilles Stoltz. Pure exploration in finitely-armed and continuous-armed bandits. *Theoretical Computer Science*, 412(19):1832–1852, 2011.

Alexandra Carpentier and Michal Valko. Simple regret for infinitely many armed bandits. In *International Conference on Machine Learning*, pp. 1133–1141. PMLR, 2015.

Olivier Chapelle and Lihong Li. An empirical evaluation of thompson sampling. In J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K.Q. Weinberger (eds.), *Advances in Neural Information Processing Systems*, volume 24. Curran Associates, Inc., 2011. URL https://proceedings.neurips.cc/paper_files/paper/2011/file/e53a0a2978c28872a4505bdb51db06dc-Paper.pdf.

Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench verified, 2024. URL https://openai.com/index/introducing-swe-bench-verified/.

Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *International conference on computers and games*, pp. 72–83. Springer, 2006.

Richard Dawkins. The evolution of evolvability. In *Artificial life*, pp. 201–220. Routledge, 2019.

Tom Everitt, Daniel Filan, Mayank Daswani, and Marcus Hutter. Self-modification of policy and utility function in rational agents. In *International conference on artificial general intelligence*, pp. 1–11. Springer, 2016.

Peter I Frazier, Warren B Powell, and Savas Dayanik. A knowledge-gradient policy for sequential information collection. *SIAM Journal on Control and Optimization*, 47(5):2410–2439, 2008.

Paul Gauthier. o1 tops aider's new polyglot leaderboard. https://aider.chat/2024/12/21/polyglot.html, dec 2024. Accessed: 2025-09-22.

Irving John Good. Speculations concerning the first ultraintelligent machine. In *Advances in computers*, volume 6, pp. 31–88. Elsevier, 1966.

John Storrs Hall. Self-improving ai: An analysis. *Minds and Machines*, 17(3):249–259, 2007.

Jesse Love Hendrikse, Trish Elizabeth Parsons, and Benedikt Hallgrímsson. Evolvability as the proper focus of evolutionary developmental biology. *Evolution & development*, 9(4):393–401, 2007.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. International Conference on Learning Representations, ICLR, 2024.

Julian Huxley. The three types of evolutionary process. *Nature*, 180(4584):454–455, 1957.

Intel. Autoround. https://github.com/intel/auto-round, 2025. GitHub repository. Accessed 2025-09-25.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Zohar Karnin, Tomer Koren, and Oren Somekh. Almost optimal exploration in multi-armed bandits. In *International conference on machine learning*, pp. 1238–1246. PMLR, 2013.

Emilie Kaufmann and Wouter M Koolen. Monte-carlo tree search by best arm identification. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

Louis Kirsch and Jürgen Schmidhuber. Self-referential meta learning. In *First Conference on Automated Machine Learning (Late-Breaking Workshop)*, 2022a.

Louis Kirsch and Jürgen Schmidhuber. Eliminating meta optimization through self-referential meta learning, 2022b. URL https://arxiv.org/abs/2212.14392.

Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European conference on machine learning*, pp. 282–293. Springer, 2006.

Robert Lange, Tom Schaul, Yutian Chen, Tom Zahavy, Valentin Dalibard, Chris Lu, Satinder Singh, and Sebastian Flennerhag. Discovering evolution strategies via meta-black-box optimization. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, pp. 29–30, 2023.

Luke Metz, C Daniel Freeman, Niru Maheswaranathan, and Jascha Sohl-Dickstein. Training learned optimizers with randomly initialized learned optimizers. *arXiv preprint arXiv:2101.07367*, 2021.

Eric Nivel, Kristinn R Thórisson, Bas R Steunebrink, Haris Dindo, Giovanni Pezzulo, Manuel Rodriguez, Carlos Hernández, Dimitri Ognibene, Jürgen Schmidhuber, Ricardo Sanz, et al. Bounded recursive self-improvement. *arXiv preprint arXiv:1312.6764*, 2013.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023.

Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent, 2025. URL https://arxiv.org/abs/2504.15228.

J. Schmidhuber. Gödel machines: self-referential universal problem solvers making provably optimal self-improvements. Technical Report IDSIA-19-03, arXiv:cs.LO/0309048, IDSIA, Manno-Lugano, Switzerland, 2003. Revised 2006.

Jürgen Schmidhuber. Evolutionary principles in self-referential learning, or on learning how to learn: The meta-meta-. hook. 1987. URL https://api.semanticscholar.org/CorpusID:264351059.

Jürgen Schmidhuber and Jieyu Zhao. Multi-agent learning with the success-story algorithm. In *Workshop on Learning in Distributed Artificial Intelligence Systems*, pp. 82–93. Springer, 1996.

Jürgen Schmidhuber, Jieyu Zhao, and Marco Wiering. Shifting inductive bias with success-story algorithm, adaptive levin search, and incremental self-improvement. *Machine Learning*, 28(1): 105–130, 1997.

Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Yizao Wang, Jean-yves Audibert, and Rémi Munos. Algorithms for infinitely many-armed bandits. In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou (eds.), *Advances in Neural Information Processing Systems*, volume 21. Curran Associates, Inc., 2008.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying llm-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. G\" odel agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation, August 2024. URL http://arxiv.org/abs/2310.02304.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents, 2025a. URL https://arxiv.org/abs/2505.22954.

Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, et al. Swe-bench goes live! *arXiv preprint arXiv:2505.23419*, 2025b.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 1592–1604, 2024.

A GÖDEL MACHINE WITH CMP ORACLE

Assumptions. The original Gödel Machine is defined in a time-aware setting, where the prover must establish not only that a proposed self-modification increases expected objectives, but also that this improvement still holds after accounting for the time required to search for proofs and compute the modification. This is necessary because, in the general case, the environment may change during these computations, and the objective is measured with respect to elapsed time.

In our setup, by contrast, the environment and the evaluation metric remain fixed throughout the agent's execution. The benchmark does not evolve over time, and the utility of any given agent is determined solely by its final performance on this static task. Importantly, we assume that the utility is measured by evaluation on tasks. It also follows the assumption of repeatable trials, meaning that the evaluation of a given agent on a task is independent of evaluation time or prior events. In other words, we are able to reset the testing environment for each test. Furthermore, we assume that the Gödel Machine prover has full knowledge of the utility function as part of its axioms of the environment. Hence, we exclude the evaluation actions from the action space. Finally, we assume that the Gödel Machine prover does not consume budget, and that the self-modifications consume an equal amount of budget—exactly one budget unit.

To summarize, we show that the CMP oracle is sufficient to imitate Gödel Machines in our specific setting that satisfies the following:

- The policy objective function is defined as a function of only the final agent, with no other rewards received before termination;
- The agent's utility is measured by its performance on evaluation tasks, under the assumption of repeatable trials: for any agent–task pair, the expected outcome is independent of evaluation time or prior events.
- The Gödel Machine prover has full knowledge of the utility function;
- The computation budget is finite and known;
- The proofs of Gödel Machines do not consume the budget;
- And each self-modification costs exactly one unit of the budget.

Proof. The proof of Gödel Machine being simulated with a CMP_π oracle is a simple observation that in the Gödel Machine setup, CMP_π is an actual state-action value function Q_π (Sutton et al., 1998) defined by the Gödel Machine, as the agents above the clades are not reachable according to the design of Gödel Machines. In order to state this, we formalize each term precisely.

We define Gödel Machine Proof search as an optimal policy on the POMDP (let's call it Gödel POMDP) defined as:

State space. Gödel POMDP operates on an extended state space, which, in addition to the tree of agents $\mathcal{T}$, consists of two special agents: a_{parent} and a_{child}, from the tree $\mathcal{T}$. For a full formal description, let's also include a remaining budget descriptor that determines the number of transitions left before the budget is exceeded.

Observation space. Observation is limited to the agents $(a_{\text{parent}}, a_{\text{child}})$ and the remaining budget b.

Action space. For most states, the action space consists of two actions—accept and reject. As in the original Gödel Machine, the action selects the new parent in the next iteration. Intuitively, accept selects a_{child} as the new parent, and reject requires the parent to remain unchanged. As the policy operates on a POMDP, it is a function of the history of the observations, which is equivalent to the state—tree $\mathcal{T}$ with special states a_{parent} and a_{child}. For the sake of simplicity in notation, we will refer to the newly selected parent agent as if it were the action. Thus, for policy π, $a_{\text{parent}} = \pi((\mathcal{T}, a_{\text{parent}}, a_{\text{child}}, b))$ corresponds to the action reject.

Scoring function. Each policy also consists of a $Score_\pi$ function that scores all possible elements in the observation. Formally, $Score_\pi : Obs \to \mathcal{T} \to \mathbb{R}$ is a function from observation into a function of agents that is returned when the budget b is equal to 0, from which the agent with the

highest score is selected as the final agent. It is a function of the observation and not the entire history, as the algorithm can only choose the final agent from the final observation.

The policy selects the highest-scoring agent, as determined by the $Score_\pi(o)$ function. Furthermore, the original Gödel Machine at the end allows only the parent or current child to be kept as the final agent. Hence, we restrict the score functions such that they always produce indicator functions of one of the observed agents, i.e., one observed agent has a score of 1, and all other agents receive a score of zero. The scoring function corresponds to the selection made by the algorithm in our original setup. After the selection dictated by the scoring function, the agent is evaluated by the Utility function U.

Transitions. Transitions generate new self-modifications from the newly selected next parent, i.e., the action output. At each transition, a self-modification of $a^{\text{new}}_{\text{parent}}$ is created and assigned as a new node a_{child}. $a^{\text{new}}_{\text{parent}}$ becomes the a_{parent} for the next time step. Formally, if $a(a)$ is the self-modification of a, and Tr is the transition function, then $Tr((\mathcal{T}, a_{\text{parent}}, a_{\text{child}}, b), a) = (\mathcal{T} \cup \{a(a)\}, a, a(a), b-1)$. The final reward in Gödel POMDP is $U(\arg\max_{a \in \mathcal{T}_B} Score_\pi(a_{\text{parent}}, a_{\text{child}})(a))$.

We also adapt the original coding setting to match the notation of Gödel POMDP. Let's call it the Extended MDP. Similarly to Gödel POMDP, its state is a tree $\mathcal{T}$, involving two special agents $a_{\text{parent}}, a_{\text{child}}$ and the budget b. The action space is the same as in our problem and consists of a single agent $a \in \mathcal{T}$ when there is still some budget left. In other cases, the policy π uses its $Score_\pi$ function to score created agents in the trees for the selection of the final agent. The transition function adds the self-modification to the tree and sets a as the new a_{parent} and $a(a)$ as the new a_{child}.

Every policy from Gödel POMDP can be mapped to a policy in the extended MDP. Specifically, let π_G be a policy in the Gödel POMDP, then we define the corresponding policy in the extended MDP π_E as $\pi_E((\mathcal{T}, a_{\text{parent}}, a_{\text{child}}, b)) = \pi_G((\mathcal{T}, a_{\text{parent}}, a_{\text{child}}, b))$. At each node, the policy chooses between the two special nodes, ignoring the rest of the tree.

Now we adapt CMP to Gödel POMDP. CMP adapted to the Gödel case is still a function of a state (and agent to expand) in the extended state space (not only observation) and the agent.

$$
\begin{aligned}
&\mathrm{CMP}_\pi((\mathcal{T}, a_p, a_c, b), a) \\
&= \mathbb{E}_{(\mathcal{T}_B, a_{Bp}, a_{Bc}, 0) \sim p_\pi(\cdot|(\mathcal{T}, a_p, a_c, b), a)} \left[U\Big(\operatorname*{arg\,max}_{a' \in C(\mathcal{T}_B, a)} Score_\pi(a_{Bp}, a_{Bc})(a') \Big) \right] \\
&\overset{(2)}{=} \mathbb{E}_{(\mathcal{T}_B, a_{Bp}, a_{Bc}, 0) \sim p_\pi(\cdot|(\mathcal{T}, a_p, a_c, b), a)} \left[U\Big(\operatorname*{arg\,max}_{a' \in \{a_{Bp}, a_{Bc}\}} Score_\pi(a_{Bp}, a_{Bc})(a') \Big) \right] \\
&\overset{(3)}{=} Q_\pi((\mathcal{T}, a_p, a_c, b), a).
\end{aligned}
$$

Where $\mathcal{T}_B$ is the final unrolled tree, a_{B_p} is the final parent and a_{B_c} is the final child in the sampled trajectory. The second equality (2) comes from the fact that the $Score_\pi$ is equal to 1 for exactly one element from the final observation -either a_{B_p} or a_{B_c}(paragraph A). The third equality (3) comes from directly unrolling the Q_π-value function.

The Gödel Machine is defined by a prover that produces a proof whether accepting or selecting a given node (or rejecting). Here, the algorithm by having access to CMP has access to the true Q value function in the Gödel POMDP. This serves as a proof of better objective of either parent or the child. Additionally, we state that we break the tie by selecting the parent node as in the original Gödel Machine. Hence, the algorithm that follows it is a Gödel Machine.

Independently, as this proof directly shows that Gödel Machine selects an action that maximizes its own Q_π-value function, it is optimal due to the Bellman Optimality Equation. With the procedure shown above, we can adapt it to the extended MDP.

□

B ALGORITHM

Algorithm 1 presents the procedure of HGM.

In the standard HGM, when an expansion ends with a new a added to the tree, the next following evaluation actions have the choice of choosing a. At the beginning of the search, the first actions have the most dense ratio of expansion actions, and the following evaluation actions would diverge more from the standard HGM since most of their choices are not generated yet, concentrating on the early finished expansions. Therefore, to avoid this bias, we initialize by expanding the initial agent 5 times with each of the processes in parallel.

The asynchronization also introduces another bias that favors agents with fewer evaluated results to be more often selected for evaluation than in the standard HGM. This is due to the fact that the easy tasks usually stop earlier than the difficult ones. This yields agents with fewer evaluations having a higher empirical mean. During our experiments, we observed that many agents were successful on the first ten evaluated tasks; however, this accuracy drops quickly as the number of evaluations grows. After having more than 50 evaluations, this bias is barely observed.

Algorithm 1 Huxley–Gödel Machine (HGM)

Input: the initial agent a_0, widening parameter α, and the percentile ϵ for final selection
Initialize a tree $\mathcal{T}$ with root a_0
Initialize counters $n^C_{\text{success}}(a), n^C_{\text{failure}}(a), n_{\text{success}}(a), n_{\text{failure}}(a)$ for all $a \in \mathcal{T}$
while Computational Budget not Exceeded **do**
  if $|\mathcal{T}| \leq n^\alpha$ and expandable parents exist **then**
    Expand:
    for each node $a \in A$ **do**
      Sample $S_C(a) \sim \text{Beta}(\tau(1 + n^C_{\text{success}}(a)),\ \tau(1 + n^C_{\text{failure}}(a)))$
    end for
    Select node $a^\star = \arg\max_a S_C(a)$
    Create child c by self-modification of $a^\star$
    Add c to $\mathcal{T}$
  else
    Evaluate:
    for each agent $a \in A$ with remaining tasks **do**
      Sample $S(a) \sim \text{Beta}(\tau(1 + n_{\text{success}}(a)),\ \tau(1 + n_{\text{failure}}(a)))$
    end for
    Select agent $a^\star = \arg\max_a S(a)$
    Allocate a benchmark task to $a^\star$
    update $n_{\text{success}}, n_{\text{failure}}$ for $a^\star$
    update $n^C_{\text{success}}, n^C_{\text{failure}}$ for $a^\star$ and ancestors
  end if
end while
Return $\text{argmax}_{a \in \mathcal{T}_B} I_\epsilon(1 + n_{\text{success}(a)}, 1 + n_{\text{failure}}(a))$

C Experimental Details

C.1 Initial Agents

Our initial agents applied in Section 4.2 are adopted from the official implementation of DGM with minor changes, including modifying API support, setting up a timeout option, and adding a length of LLM interaction restriction. The initial agent is essentially a single loop of LLM queries with two tool options, i.e., file editing and bash command execution. We set a time limit of one hour for each agent execution.

The initial agents used in SWE-bench experiments and Polyglot experiments differ in that the Polyglot initial agent includes test commands with different programming language support. There are two additional functions in the SWE-bench initial agent that serve to summarize existing tests and execute the tests with a report generated, respectively.

The initial agent employed in Section 4.3 is further adjusted by removing the file-editing tool, leaving only the bash tool, to minimize initial inductive bias. The time limit is extended to five hours for both self-modification and task evaluation, reducing the risk of prematurely eliminating stronger agents due to time constraints.

C.2 Other Details

For the Polyglot experiments presented in Section 4.2, the exact large language model used for self-modification is an int4 and int8 mixed quantized version of Qwen3-Coder-480B-A35B-Instruct generated by AutoRound (Intel, 2025). Overall, we spent approximately $5000 USD to produce the experimental results, including all three methods.

D EMPIRICAL CMP AND ITS ESTIMATION

In this section, we provide the exact formula to compute the empirical CMP and the variant of our CMP estimator being used in Section 4.1 for correlation analysis. The empirical CMP of an agent a as a node in a tree is defined as

$$\max_{a' \in C(a) \setminus \{a\}} \frac{n_{\text{success}}(a')}{n_{\text{success}}(a') + n_{\text{failure}}(a')}.$$

The prediction of our CMP estimator is defined as

$$\frac{n^C_{\text{success}}(a) - n_{\text{success}}(a) - n^C_{\text{success}}(b^*)}{n^C_{\text{failure}}(a) - n_{\text{failure}}(a) - n^C_{\text{failure}}(b^*) + n^C_{\text{success}}(a) - n_{\text{success}}(a) - n^C_{\text{success}}(b^*)},$$

where b^* is a child of a such that

$$\left(\text{argmax}_{n \in C(a)} \frac{n_{\text{success}}(n)}{n_{\text{failure}}(n)} \right) \cap C(b^*) \neq \emptyset.$$

For both SICA and DGM, we consider the benchmark performance of an agent as their estimator of the agent's CMP.

E BASELINES

Table 2 summarizes the three subpolicies of SICA, DGM, and HGM, which define solutions to the iterative tree search problem defined in 2.

Subpolicy	**SICA**	**DGM**	**HGM (Ours)**
Selection Policy	Alternates between modification and evaluation.	Alternates between modification and evaluation.	Adaptive choice between modification and evaluation.
Expansion Policy	Greedily selects the agent with the best performance up to this point and modifies it with the entire history accessible to the agent.	Selects the node probabilistically based on the evaluation metric and the number of children of the agents.	Selects the node based on the statistics of the *clade* stemming from a given node.
Evaluation Policy	Evaluates the most recently created agent on the entire evaluation dataset.	Progressively evaluates the last created agent on subsets of the dataset, expanding if results are promising.	Selects the agent based on the statistics and evaluates it on a single task.

Table 5: Comparison of structured policies across self-improving methods. Each method is described by three subpolicies: Selection Policy, Expansion Policy, and Evaluation Policy.

F Discovered Agents

We present interesting findings about HGM-discovered agents during our experiments. By manually inspecting the diff patch files that were generated by self-modification, we make interesting observations.

One engaging self-modification we found implements iterative refinement to make incremental improvements. In the self-improving context, it means that during one expansion (self-modification) step, the agent is accurately performing multiple self-modifications. More interestingly, this phenomenon can be viewed as the agent being self-motivated to perform self-modification, instead of being instructed to do so by humans. This self-motivated repetitive self-modification automatically creates arbitrary levels of meta-improvement in the sense that each self-modification ideally needs to consider all its subsequent self-modifications, which the subsequent self-modifications also need to consider future self-modifications. We show in List 1 the part of the patch file that includes the implementation of this self-modification.

Another interesting phenomenon we observe is the file structures that self-modifying agents create. Many agents tend to write the chunk they self-modify into a diff patch file, usually with a constant file name. When multiple consecutive attempts of these self-modifying actions apply, the structure of these git diff files becomes abstruse. Imagine that there is a diff file that includes certain edits. Once there are changes applied to this file, it then generates the diff patch file. In this new patch file, a nested format of editing record appears. At the beginning of some lines, there are multiple + and - signs that correspond to nested file edits. This is a natural format to present information that inspires long-sighted self-improvement since it shows how self-modifications indeed influence subsequent self-modifications. The authors find it mind-bending to understand such a file structure and parse the semantics manually. For readers interested in further investigation, we provide one instance of such a file structure in List 2.

Listing 1: The self-modifying agent that executes multiple self-modifications per instruction.

```
+def attempt_error_resolution(git_dir, test_output, test_error,
    ↪ language):
+    """
+    Attempt to automatically diagnose and resolve errors.
+    Returns a tuple of (resolved, message) where resolved indicates if
    ↪ errors were fixed.
+    """
+    safe_log("Attempting automated error diagnosis and resolution...")
+
+    # Diagnose errors using our enhanced bash tool function
+    diagnosis = diagnose_errors(test_output, test_error, "")
+
+    if not diagnosis["has_errors"]:
+        return False, "No errors detected to resolve."
+
+    resolution_messages = []
+
+    # Try to apply automated fixes for each diagnosed error
+    for error in diagnosis["errors"]:
+        safe_log(f"Processing error: {error['type']} -
    ↪ {error['description']}")
+
+        # Simple resolution strategies based on error type
+        if error["type"] == "python_module_not_found":
+            # For Python module not found errors, we might install the
    ↪ module
+            match = re.search(r"No module named '([^']+)'",
    ↪ error["description"])
+            if match:
+                module = match.group(1)
+                resolution_messages.append(f"Would attempt to install
    ↪ Python module: {module}")
+                # In practice, we would run: pip install {module}
```

```
+                # But we'll skip actual installation to avoid side
    ↪ effects
+
+        elif error["type"] == "python_syntax_error" and "file" in error:
+            # For syntax errors, we could potentially apply fixes
+            file_path = os.path.join(git_dir, error["file"])
+            if os.path.exists(file_path):
+                resolution_messages.append(f"Would attempt to fix
    ↪ syntax error in {file_path} at line {error.get('line',
    ↪ 'unknown')}")
+                # In practice, we would use the editor tool's apply_fix
    ↪ command
+                # This is just a demonstration of what could be done
+
+        elif error["type"] == "test_failure":
+            # For test failures, we might suggest reviewing the
    ↪ implementation
+            resolution_messages.append("Would analyze test failures and
    ↪ suggest implementation improvements")
+
+    if resolution_messages:
+        return True, "Automated resolution attempted:\n" +
    ↪ "\n".join(resolution_messages)
+    else:
+        return False, "No automated resolutions available for detected
    ↪ errors."
+
class AgenticSystem:
     def __init__(
             self,
@@ -243,6 +293,16 @@ Your task is to make changes to the files in the
    ↪ {self.git_dir} directory to add

             safe_log(f"Attempt {attempt + 1} test results: {'PASSED' if
    ↪ test_success else 'FAILED'}")

+            # If tests failed, attempt automated error resolution
+            if not test_success:
+                resolved, resolution_message = attempt_error_resolution(
+                    self.git_dir, test_output, test_error, self.language
+                )
+                safe_log(f"Error resolution: {resolution_message}")
+
+                # Even if we couldn't automatically resolve, we still
    ↪ provide feedback
+                # In a more advanced implementation, we might actually
    ↪ apply fixes here
+
             # If this is the first attempt or tests passed and we
    ↪ didn't have a successful attempt yet, update best patch
             if attempt == 0 or (test_success and (best_patch is None or
    ↪ not best_test_results)):
                 best_patch = current_patch
@@ -278,37 +338,31 @@ Please revise your code to fix these issues and
    ↪ try again.
         # Log final summary
         safe_log(f"\n{'='*20} FINAL SUMMARY {'='*20}")
         safe_log(f"Best solution found on attempt:
    ↪ {best_test_results['attempt'] if best_test_results else 'None'}")
-        safe_log(f"Tests passed: {best_test_results['test_success'] if
    ↪ best_test_results else 'Unknown'}")
+        safe_log(f"Final test result: {'PASSED' if best_test_results
    ↪ and best_test_results['test_success'] else 'FAILED'}")
+
+        if best_test_results:
```

```
+            safe_log(f"Final test
    ↪ output:\n{best_test_results['test_output']}")
+            if best_test_results['test_error']:
+                safe_log(f"Final test
    ↪ errors:\n{best_test_results['test_error']}")

-        # Save attempt history to a file
-        history_file =
    ↪ os.path.join(os.path.dirname(self.chat_history_file),
    ↪ 'attempt_history.md')
-        with open(history_file, 'w') as f:
-            f.write("# Attempt History\n\n")
-            for result in self.attempt_history:
-                f.write(f"## Attempt {result['attempt']}\n")
-                f.write(f"**Tests Passed**: {result['test_success']}\n")
-                f.write(f"**LLM Calls Used**: {result['llm_calls']}\n")
-                f.write(f"**Test
    ↪ Output**:\n```\n{result['test_output']}\n```\n")
-                f.write(f"**Test
    ↪ Error**:\n```\n{result['test_error']}\n```\n")
-                f.write(f"**Patch**:\n```\n{result['patch']}\n```\n\n")
+        return bool(best_test_results and
    ↪ best_test_results['test_success'])
```

Listing 2: An instance of the nested diff patch format.

```
diff --git a/attempt_history.md b/attempt_history.md
new file mode 100644
index 0000000..b132b1a
--- /dev/null
+++ b/attempt_history.md
@@ -0,0 +1,727 @@
+# Attempt History
+
+## Attempt 1
+**Tests Passed**: True
+**LLM Calls Used**: 18
+**Test Output**:
+```
+============================= test session starts
    ↪ ==============================
+platform linux -- Python 3.10.18, pytest-8.4.2, pluggy-1.6.0 --
    ↪ /usr/local/bin/python3.10
+cachedir: .pytest_cache
+rootdir: /dgm
+configfile: pytest.ini
+testpaths: tests
+plugins: asyncio-1.1.0, anyio-4.10.0
+asyncio: mode=strict, asyncio_default_fixture_loop_scope=None,
    ↪ asyncio_default_test_loop_scope=function
+collecting ... collected 29 items
+
+tests/test_bash_tool.py::TestBashTool::test_simple_command PASSED
    ↪  [  3%]
+tests/test_bash_tool.py::TestBashTool::test_multiple_commands PASSED
    ↪  [  6%]
+tests/test_bash_tool.py::TestBashTool::test_command_with_error PASSED
    ↪  [ 10%]
+tests/test_bash_tool.py::TestBashTool::test_environment_variables
    ↪ PASSED [ 13%]
+tests/test_bash_tool.py::TestBashTool::test_command_output_processing
    ↪ PASSED [ 17%]
+tests/test_bash_tool.py::TestBashTool::test_long_running_command PASSED
    ↪  [ 20%]
```

```
+tests/test_bash_tool.py::TestBashTool::test_invalid_commands[invalid_command_name]
    ↪ PASSED [ 24%]
+tests/test_bash_tool.py::TestBashTool::test_invalid_commands[cd
    ↪ /nonexistent/path] PASSED [ 27%]
+tests/test_bash_tool.py::TestBashTool::test_invalid_commands[/bin/nonexistent]
    ↪ PASSED [ 31%]
+tests/test_bash_tool.py::TestBashTool::test_command_with_special_chars
    ↪ PASSED [ 34%]
+tests/test_bash_tool.py::TestBashTool::test_multiple_line_output PASSED
    ↪  [ 37%]
+tests/test_bash_tool.py::TestBashTool::test_large_output_handling
    ↪ PASSED [ 41%]
+tests/test_edit_tool.py::TestEditorTool::test_view_file PASSED
    ↪  [ 44%]
+tests/test_edit_tool.py::TestEditorTool::test_create_file PASSED
    ↪  [ 48%]
+tests/test_edit_tool.py::TestEditorTool::test_create_existing_file
    ↪ PASSED [ 51%]
+tests/test_edit_tool.py::TestEditorTool::test_edit_file PASSED
    ↪  [ 55%]
+tests/test_edit_tool.py::TestEditorTool::test_edit_nonexistent_file
    ↪ PASSED [ 58%]
+tests/test_edit_tool.py::TestEditorTool::test_view_directory PASSED
    ↪  [ 62%]
+tests/test_edit_tool.py::TestEditorTool::test_invalid_path PASSED
    ↪  [ 65%]
+tests/test_edit_tool.py::TestEditorTool::test_invalid_commands[unknown_command]
    ↪ PASSED [ 68%]
+tests/test_edit_tool.py::TestEditorTool::test_invalid_commands[] PASSED
    ↪  [ 72%]
+tests/test_edit_tool.py::TestEditorTool::test_invalid_commands[None]
    ↪ PASSED [ 75%]
+tests/test_error_diagnosis.py::TestErrorDiagnosis::test_python_syntax_error_diagnosis
    ↪ PASSED [ 79%]
+tests/test_error_diagnosis.py::TestErrorDiagnosis::test_python_module_not_found_diagnosis
    ↪ PASSED [ 82%]
+tests/test_error_diagnosis.py::TestErrorDiagnosis::test_no_error_diagnosis
    ↪ PASSED [ 86%]
+tests/test_error_diagnosis.py::TestErrorDiagnosis::test_format_diagnosis_with_errors
    ↪ PASSED [ 89%]
+tests/test_error_diagnosis.py::TestErrorDiagnosis::test_format_diagnosis_without_errors
    ↪ PASSED [ 93%]
+tests/test_error_diagnosis.py::TestAutomatedFixes::test_apply_missing_import_fix
    ↪ PASSED [ 96%]
+tests/test_error_diagnosis.py::TestAutomatedFixes::test_apply_syntax_error_fix
    ↪ PASSED [100%]
+
+=================================== PASSES
    ↪ ===================================
+=========================== short test summary info
    ↪ ============================
+PASSED tests/test_bash_tool.py::TestBashTool::test_simple_command
+PASSED tests/test_bash_tool.py::TestBashTool::test_multiple_commands
+PASSED tests/test_bash_tool.py::TestBashTool::test_command_with_error
+PASSED tests/test_bash_tool.py::TestBashTool::test_environment_variables
+PASSED
    ↪ tests/test_bash_tool.py::TestBashTool::test_command_output_processing
+PASSED tests/test_bash_tool.py::TestBashTool::test_long_running_command
+PASSED
    ↪ tests/test_bash_tool.py::TestBashTool::test_invalid_commands[invalid_command_name]
+PASSED tests/test_bash_tool.py::TestBashTool::test_invalid_commands[cd
    ↪ /nonexistent/path]
+PASSED
    ↪ tests/test_bash_tool.py::TestBashTool::test_invalid_commands[/bin/nonexistent]
```

```
+PASSED
    ↪ tests/test_bash_tool.py::TestBashTool::test_command_with_special_chars
+PASSED tests/test_bash_tool.py::TestBashTool::test_multiple_line_output
+PASSED tests/test_bash_tool.py::TestBashTool::test_large_output_handling
+PASSED tests/test_edit_tool.py::TestEditorTool::test_view_file
+PASSED tests/test_edit_tool.py::TestEditorTool::test_create_file
+PASSED
    ↪ tests/test_edit_tool.py::TestEditorTool::test_create_existing_file
+PASSED tests/test_edit_tool.py::TestEditorTool::test_edit_file
+PASSED
    ↪ tests/test_edit_tool.py::TestEditorTool::test_edit_nonexistent_file
+PASSED tests/test_edit_tool.py::TestEditorTool::test_view_directory
+PASSED tests/test_edit_tool.py::TestEditorTool::test_invalid_path
+PASSED
    ↪ tests/test_edit_tool.py::TestEditorTool::test_invalid_commands[unknown_command]
+PASSED tests/test_edit_tool.py::TestEditorTool::test_invalid_commands[]
+PASSED
    ↪ tests/test_edit_tool.py::TestEditorTool::test_invalid_commands[None]
+PASSED
    ↪ tests/test_error_diagnosis.py::TestErrorDiagnosis::test_python_syntax_error_diagnosis
+PASSED
    ↪ tests/test_error_diagnosis.py::TestErrorDiagnosis::test_python_module_not_found_diagnosi
+PASSED
    ↪ tests/test_error_diagnosis.py::TestErrorDiagnosis::test_no_error_diagnosis
+PASSED
    ↪ tests/test_error_diagnosis.py::TestErrorDiagnosis::test_format_diagnosis_with_errors
+PASSED
    ↪ tests/test_error_diagnosis.py::TestErrorDiagnosis::test_format_diagnosis_without_errors
+PASSED
    ↪ tests/test_error_diagnosis.py::TestAutomatedFixes::test_apply_missing_import_fix
+PASSED
    ↪ tests/test_error_diagnosis.py::TestAutomatedFixes::test_apply_syntax_error_fix
+============================== 29 passed in 3.58s
    ↪ ==============================
+
+```
+**Test Error**:
+```
+
+```
+**Patch**:
+```
+diff --git a/coding_agent.py b/coding_agent.py
+index 78e8ad4..77e5097 100644
+--- a/coding_agent.py
++++ b/coding_agent.py
+@@ -5,9 +5,13 @@ from logging.handlers import RotatingFileHandler
+ import os
+ import threading
+ import time
++import json
++import re
+
+ from llm_withtools import CLAUDE_MODEL, OPENAI_MODEL, chat_with_agent
+ from utils.git_utils import diff_versus_commit, reset_to_commit,
    ↪ apply_patch
++from tools.bash import diagnose_errors
++from tools.edit import apply_automated_fix, read_file, write_file
+
+ # reset_to_commit(git_dname, commit)
+ # apply_patch(git_dname, patch_str)
+@@ -136,6 +140,52 @@ def run_tests(git_dir, language):
+         # Always change back to original directory
+         os.chdir(original_cwd)
+
```

```
++def attempt_error_resolution(git_dir, test_output, test_error,
    ↪ language):
++    """
++    Attempt to automatically diagnose and resolve errors.
++    Returns a tuple of (resolved, message) where resolved indicates if
    ↪ errors were fixed.
++    """
++    """
++    safe_log("Attempting automated error diagnosis and resolution...")
++
++    # Diagnose errors using our enhanced bash tool function
++    diagnosis = diagnose_errors(test_output, test_error, "")
++
++    if not diagnosis["has_errors"]:
++        return False, "No errors detected to resolve."
++
++    resolution_messages = []
++
++    # Try to apply automated fixes for each diagnosed error
++    for error in diagnosis["errors"]:
++        safe_log(f"Processing error: {error['type']} -
    ↪ {error['description']}")
++
++        # Simple resolution strategies based on error type
++        if error["type"] == "python_module_not_found":
++            # For Python module not found errors, we might install the
    ↪ module
++            match = re.search(r"No module named '([^']+)'",
    ↪ error["description"])
++            if match:
++                module = match.group(1)
++                resolution_messages.append(f"Would attempt to install
    ↪ Python module: {module}")
++                # In practice, we would run: pip install {module}
++                # But we'll skip actual installation to avoid side
    ↪ effects
++
++        elif error["type"] == "python_syntax_error" and "file" in
    ↪ error:
++            # For syntax errors, we could potentially apply fixes
++            file_path = os.path.join(git_dir, error["file"])
++            if os.path.exists(file_path):
++                resolution_messages.append(f"Would attempt to fix
    ↪ syntax error in {file_path} at line {error.get('line',
    ↪ 'unknown')}")
++                # In practice, we would use the editor tool's
    ↪ apply_fix command
++                # This is just a demonstration of what could be done
++
++        elif error["type"] == "test_failure":
++            # For test failures, we might suggest reviewing the
    ↪ implementation
++            resolution_messages.append("Would analyze test failures
    ↪ and suggest implementation improvements")
++
++    if resolution_messages:
++        return True, "Automated resolution attempted:\n" +
    ↪ "\n".join(resolution_messages)
++    else:
++        return False, "No automated resolutions available for detected
    ↪ errors."
++
+ class AgenticSystem:
+     def __init__(
+             self,
```

```
+@@ -243,6 +293,16 @@ Your task is to make changes to the files in the
    ↪ {self.git_dir} directory to add
+
+             safe_log(f"Attempt {attempt + 1} test results: {'PASSED'
    ↪ if test_success else 'FAILED'}")
+
++            # If tests failed, attempt automated error resolution
++            if not test_success:
++                resolved, resolution_message =
    ↪ attempt_error_resolution(
++                    self.git_dir, test_output, test_error,
    ↪ self.language
++                )
++                safe_log(f"Error resolution: {resolution_message}")
++
++                # Even if we couldn't automatically resolve, we still
    ↪ provide feedback
++                # In a more advanced implementation, we might actually
    ↪ apply fixes here
++
+             # If this is the first attempt or tests passed and we
    ↪ didn't have a successful attempt yet, update best patch
+             if attempt == 0 or (test_success and (best_patch is None
    ↪ or not best_test_results)):
+                 best_patch = current_patch
+@@ -278,37 +338,31 @@ Please revise your code to fix these issues and
    ↪ try again.
+         # Log final summary
+         safe_log(f"\n{'='*20} FINAL SUMMARY {'='*20}")
+         safe_log(f"Best solution found on attempt:
    ↪ {best_test_results['attempt'] if best_test_results else 'None'}")
+-        safe_log(f"Tests passed: {best_test_results['test_success'] if
    ↪ best_test_results else 'Unknown'}")
++        safe_log(f"Final test result: {'PASSED' if best_test_results
    ↪ and best_test_results['test_success'] else 'FAILED'}")
++
++        if best_test_results:
++            safe_log(f"Final test
    ↪ output:\n{best_test_results['test_output']}")
++            if best_test_results['test_error']:
++                safe_log(f"Final test
    ↪ errors:\n{best_test_results['test_error']}")
+
+-        # Save attempt history to a file
+-        history_file =
    ↪ os.path.join(os.path.dirname(self.chat_history_file),
    ↪ 'attempt_history.md')
+-        with open(history_file, 'w') as f:
+-            f.write("# Attempt History\n\n")
+-            for result in self.attempt_history:
+-                f.write(f"## Attempt {result['attempt']}\n")
+-                f.write(f"**Tests Passed**:
    ↪ {result['test_success']}\n")
+-                f.write(f"**LLM Calls Used**: {result['llm_calls']}\n")
+-                f.write(f"**Test
    ↪ Output**:\n```\n{result['test_output']}\n```\n")
+-                f.write(f"**Test
    ↪ Error**:\n```\n{result['test_error']}\n```\n")
+-                f.write(f"**Patch**:\n```\n{result['patch']}\n```\n\n")
++        return bool(best_test_results and
    ↪ best_test_results['test_success'])
+
+-def main():
+-    parser = argparse.ArgumentParser(description='Process repository
    ↪ with an agentic system.')
```

```
+-    parser.add_argument('--problem_statement', required=True,
    ↪ help='The problem statement to process')
+-    parser.add_argument('--git_dir', required=True, help='Path to git
    ↪ repository directory')
+-    parser.add_argument('--base_commit', required=True, help='Base
    ↪ commit hash to compare against')
+-    parser.add_argument('--chat_history_file', required=True,
    ↪ help='Path to chat history file')
+-    parser.add_argument('--outdir', required=False, default="/dgm/",
    ↪ help='Output directory')
+-    parser.add_argument('--test_description', default=None,
    ↪ required=False, help='Description of how to test the repository')
+-    parser.add_argument('--self_improve', default=False,
    ↪ action='store_true', help='Whether to self-improve the repository
    ↪ or solving swe')
+-    parser.add_argument('--language', required=False,
    ↪ default="python", choices=['cpp', 'java', 'python', 'go', 'rust',
    ↪ 'javascript'], help='Task\'s programming language')
+-    parser.add_argument('--model', required=False,
    ↪ default=CLAUDE_MODEL, help='LLM model to use for processing')
+-    parser.add_argument('--timeout', type=int, default=3600,
    ↪ help='Timeout for LLM calls in seconds')
+-    parser.add_argument('--max_attempts', type=int, default=3,
    ↪ help='Maximum number of solution attempts')
++if __name__ == "__main__":
++    parser = argparse.ArgumentParser(description="Run the Agentic
    ↪ System on a coding task.")
++    parser.add_argument("--problem_statement", type=str,
    ↪ required=True, help="Problem statement")
++    parser.add_argument("--git_dir", type=str, required=True,
    ↪ help="Git repository directory")
++    parser.add_argument("--base_commit", type=str, required=True,
    ↪ help="Base commit hash")
++    parser.add_argument("--chat_history_file", type=str,
    ↪ default="./chat_history.md", help="Chat history file")
++    parser.add_argument("--test_description", type=str, help="Test
    ↪ description")
++    parser.add_argument("--self_improve", action="store_true",
    ↪ help="Enable self-improvement mode")
++    parser.add_argument("--language", type=str, default="python",
    ↪ help="Programming language")
++    parser.add_argument("--model", type=str, default=OPENAI_MODEL,
    ↪ help="Model to use")
++    parser.add_argument("--max_attempts", type=int, default=3,
    ↪ help="Maximum number of attempts")
++    parser.add_argument("--timeout", type=int, default=600,
    ↪ help="Timeout for each attempt")
++
+     args = parser.parse_args()
+-
+-    # Process the repository
+-    agentic_system = AgenticSystem(
++
++    system = AgenticSystem(
+         problem_statement=args.problem_statement,
+         git_dir=args.git_dir,
+         base_commit=args.base_commit,
+@@ -319,15 +373,7 @@ def main():
+         model=args.model,
+         max_attempts=args.max_attempts
+     )
+-
+-    # Run the agentic system to try to solve the problem
+-    agentic_system.forward(args.timeout)
+-
```

```
+-    # Get code diff and save to model_patch.diff
+-    model_patch = diff_versus_commit(args.git_dir, args.base_commit)
+-    model_patch_outfile = os.path.join(args.outdir,
    ↪ 'model_patch.diff') if args.outdir else 'model_patch.diff'
+-    with open(model_patch_outfile, 'w') as f:
+-        f.write(model_patch)
+-
+-if __name__ == "__main__":
+-    main()
+\ No newline at end of file
++
++    success = system.forward(timeout=args.timeout)
++    exit_code = 0 if success else 1
++    exit(exit_code)
+\ No newline at end of file
+diff --git a/tests/test_error_diagnosis.py
    ↪ b/tests/test_error_diagnosis.py
+new file mode 100644
+index 0000000..5beffbe
+--- /dev/null
++++ b/tests/test_error_diagnosis.py
+@@ -0,0 +1,98 @@
++import pytest
++from tools.bash import diagnose_errors, format_diagnosis
++from tools.edit import apply_automated_fix
++
++class TestErrorDiagnosis:
++    def test_python_syntax_error_diagnosis(self):
++        """Test diagnosis of Python syntax errors."""
++        output = ""
++        error = '''File "test.py", line 3
++    print("Hello World"
++                       ^
++SyntaxError: unexpected EOF while parsing'''
++
++        diagnosis = diagnose_errors(output, error, "python test.py")
++        assert diagnosis["has_errors"] is True
++        assert len(diagnosis["errors"]) == 1
++        assert diagnosis["errors"][0]["type"] == "python_syntax_error"
++        assert "SyntaxError" in diagnosis["errors"][0]["description"]
++
++    def test_python_module_not_found_diagnosis(self):
++        """Test diagnosis of Python module not found errors."""
++        output = ""
++        error = "ModuleNotFoundError: No module named
    ↪ 'nonexistent_module'"
++
++        diagnosis = diagnose_errors(output, error, "python test.py")
++        assert diagnosis["has_errors"] is True
++        assert len(diagnosis["errors"]) == 1
++        assert diagnosis["errors"][0]["type"] ==
    ↪ "python_module_not_found"
++        assert "nonexistent_module" in
    ↪ diagnosis["errors"][0]["description"]
++
++    def test_no_error_diagnosis(self):
++        """Test diagnosis when there are no errors."""
++        output = "Success!"
++        error = ""
++
++        diagnosis = diagnose_errors(output, error, "echo Success")
++        assert diagnosis["has_errors"] is False
++        assert len(diagnosis["errors"]) == 0
++
++    def test_format_diagnosis_with_errors(self):
```

```
++        """Test formatting of diagnosis with errors."""
++        diagnosis = {
++            "has_errors": True,
++            "errors": [
++                {
++                    "type": "test_error",
++                    "description": "Test error description",
++                    "suggestions": ["Suggestion 1", "Suggestion 2"]
++                }
++            ]
++        }
++
++        formatted = format_diagnosis(diagnosis)
++        assert "Automated Error Diagnosis" in formatted
++        assert "Test error description" in formatted
++        assert "Suggestion 1" in formatted
++
++    def test_format_diagnosis_without_errors(self):
++        """Test formatting of diagnosis without errors."""
++        diagnosis = {
++            "has_errors": False,
++            "errors": []
++        }
++
++        formatted = format_diagnosis(diagnosis)
++        assert "No errors detected" in formatted
++
++class TestAutomatedFixes:
++    def test_apply_missing_import_fix(self):
++        """Test applying a missing import fix."""
++        content = """def hello():
++    print(json.dumps({"message": "hello"}))
++"""
++
++        fix_info = {
++            "type": "missing_import",
++            "module": "json",
++            "description": "Added missing import for json module"
++        }
++
++        fixed_content = apply_automated_fix(content, fix_info)
++        assert "import json" in fixed_content
++        assert "def hello():" in fixed_content
++
++    def test_apply_syntax_error_fix(self):
++        """Test applying a syntax error fix."""
++        content = """def hello()
++    print("Hello World")
++"""
++
++        fix_info = {
++            "type": "syntax_error_fix",
++            "line": 1,
++            "description": "Fixed syntax error"
++        }
++
++        fixed_content = apply_automated_fix(content, fix_info)
++        assert "Fixed syntax" in fixed_content
+\ No newline at end of file
```